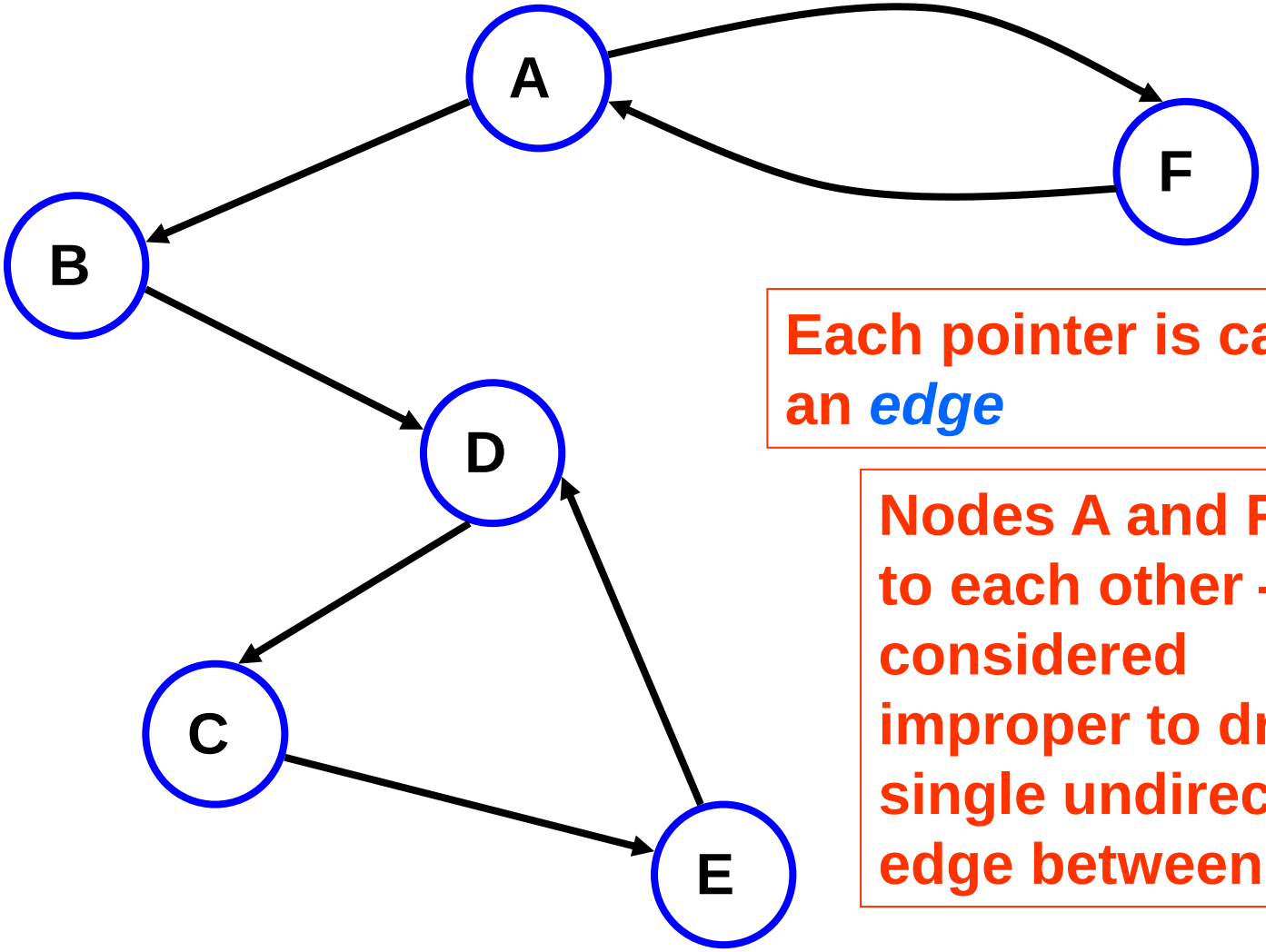
Lecture 08a
Graphs

Graphs

- Graphs are a very general data structure
- A graph consists of a set of nodes called ***vertices***
- Any vertex can point to any number of other vertices
- It is possible that no vertex in the graph points to any other vertex
- Each vertex may point to every other vertex, even if there are thousands of vertices

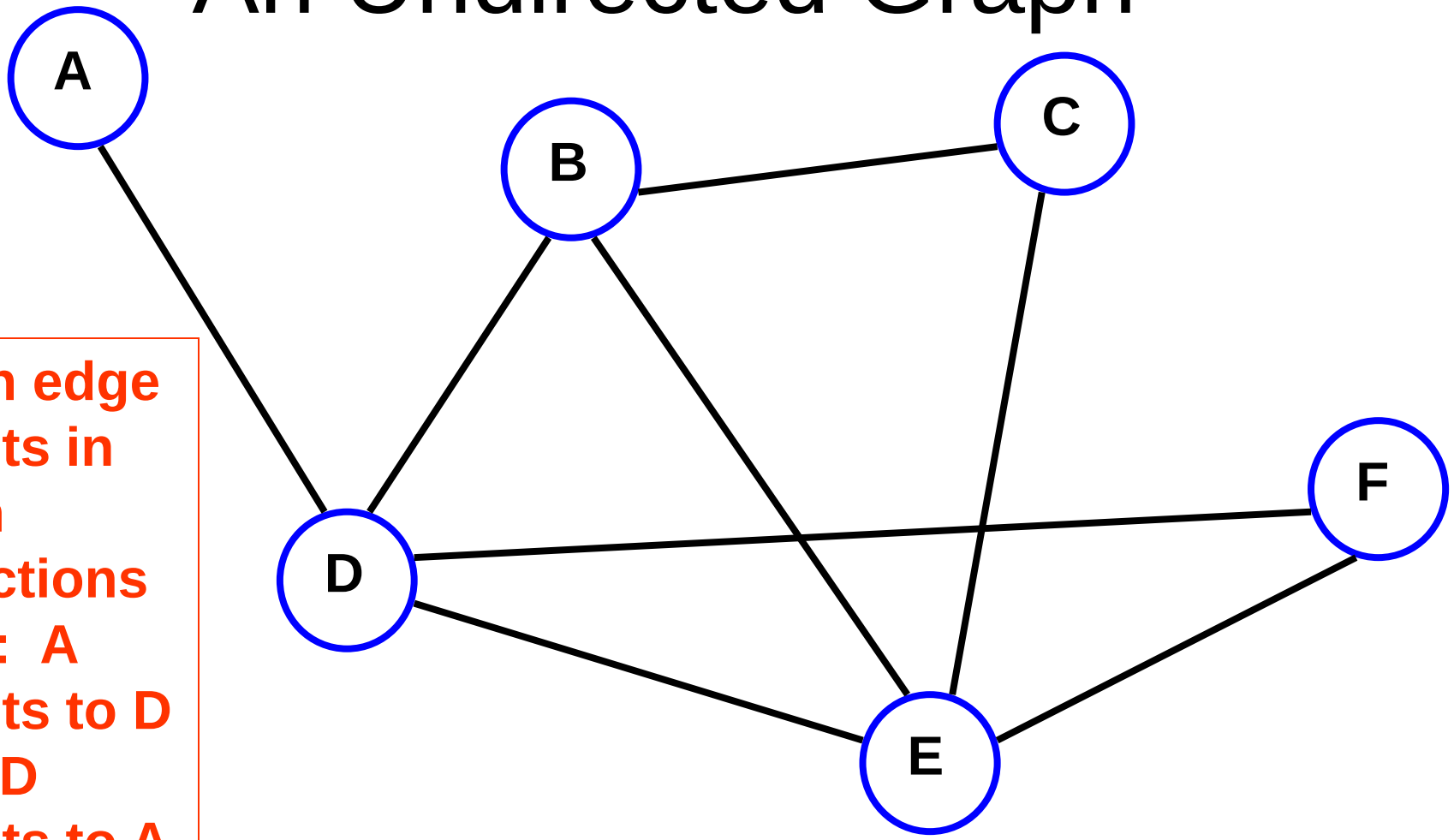
A Directed Graph (Digraph)



Each pointer is called an *edge*

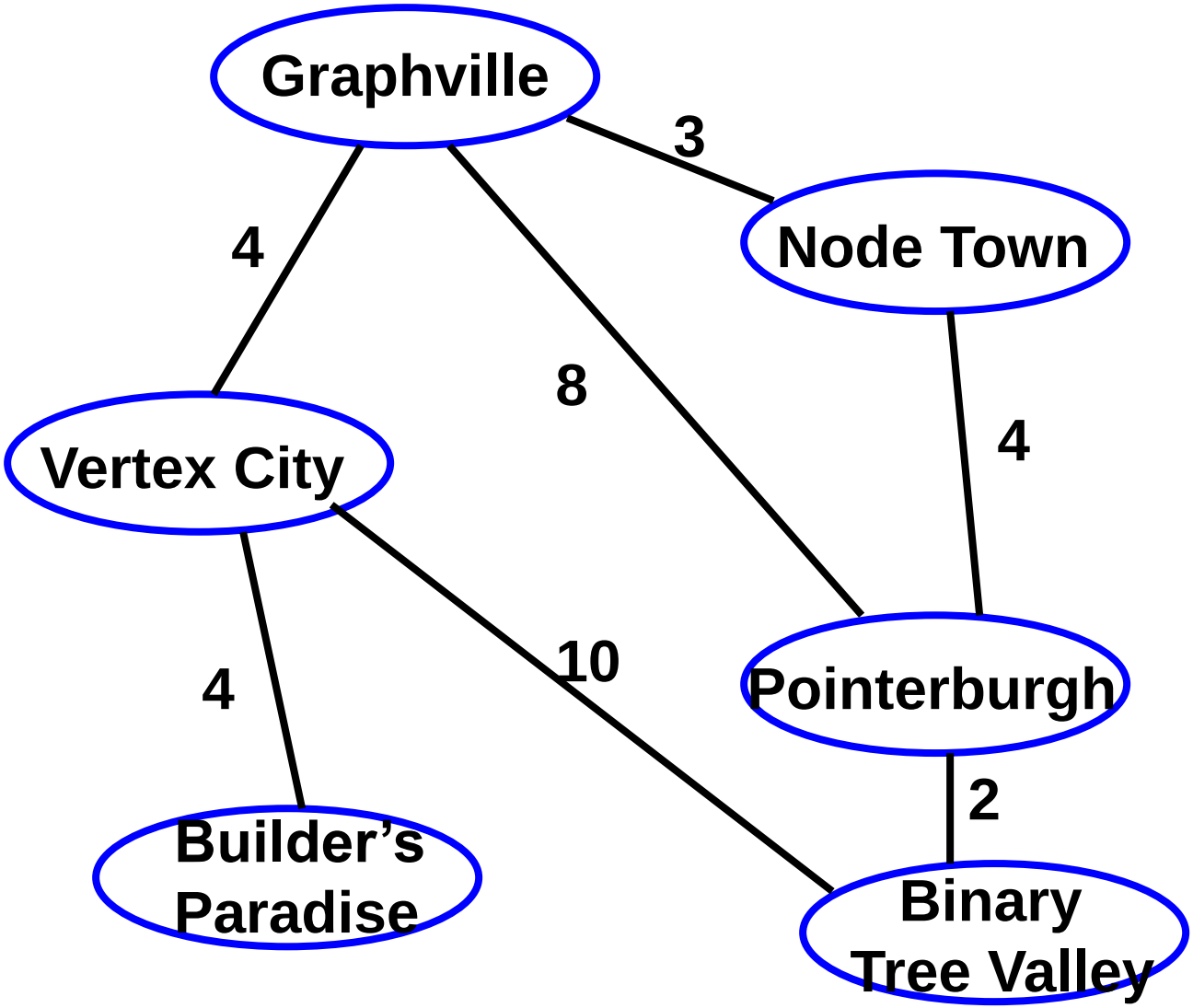
Nodes A and F point to each other – it is considered improper to draw a single undirected edge between them

An Undirected Graph



Each edge points in both directions – ex: A points to D and D points to A

Undirected Weighted Graph



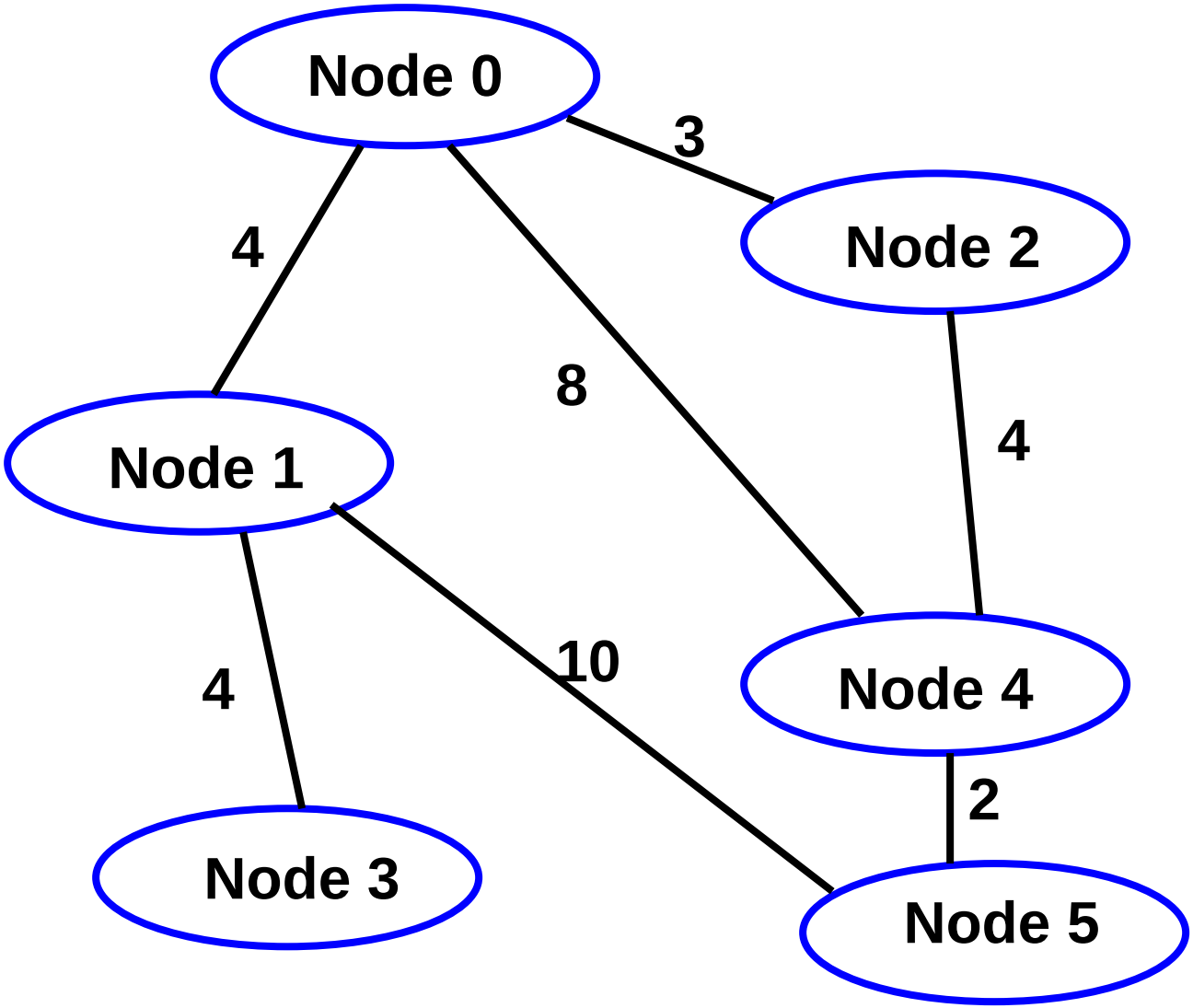
Graph Implementation

- A vertex A is said to be ***adjacent*** to a vertex B if there is an edge pointing from A to B
- Graphs can be implemented in a couple of popular ways:
  - adjacency matrix
  - adjacency list

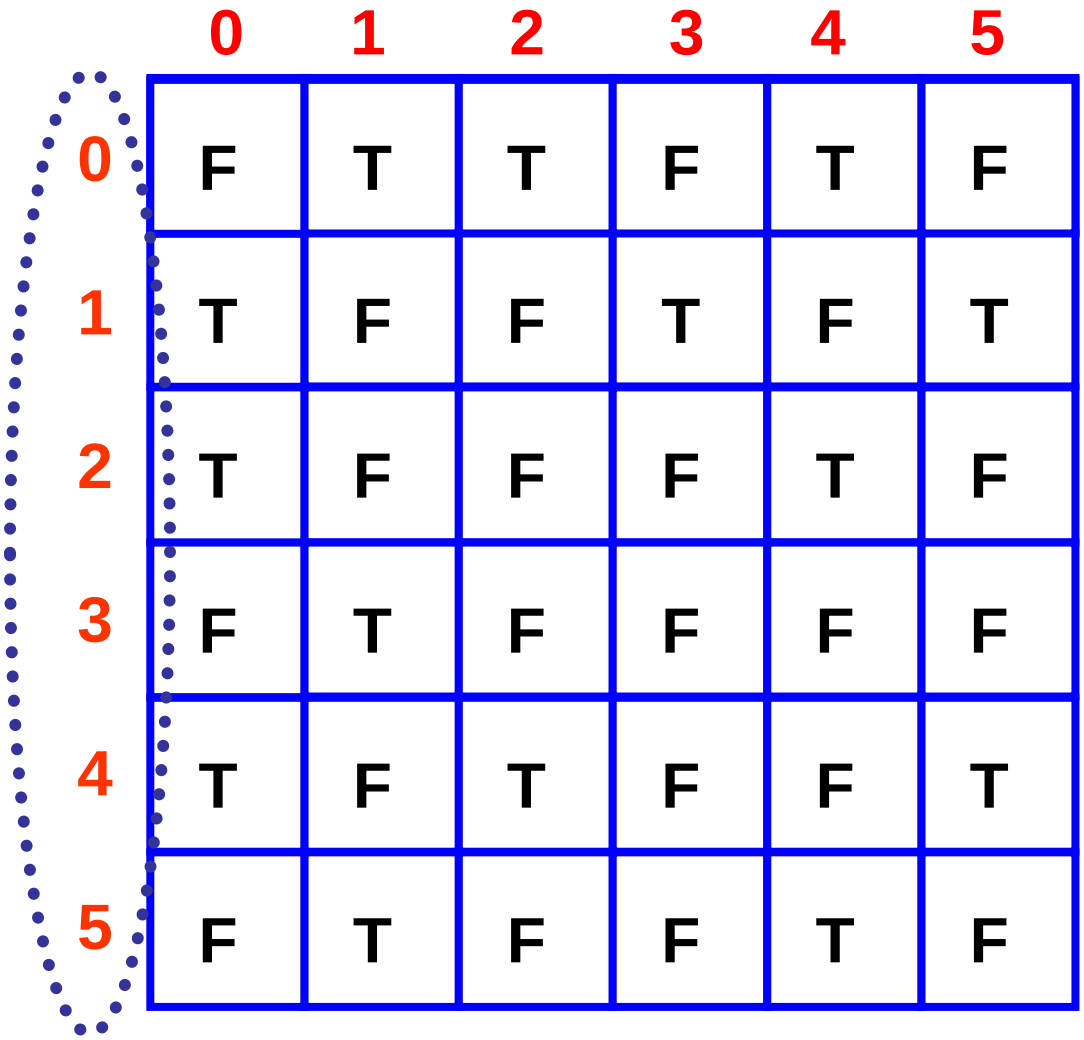
Adjacency Matrix

- An adjacency matrix is a 2-dimensional array.
- Nodes are given a key from 0 to n – 1
- If weight is not important, the adjacency matrix is a 2-dimensional array of Boolean (true or false) type variables
- If weight is important, the adjacency matrix is a 2-dimensional array of integer type variables

Undirected Weighted Graph



Adjacency Matrix (cont.)



	0	1	2	3	4	5
0	F	T	T	F	T	F
1	T	F	F	T	F	T
2	T	F	F	F	T	F
3	F	T	F	F	F	F
4	T	F	T	F	F	T
5	F	T	F	F	T	F

Undirected graph

The row numbers give the vertex number of the vertex that an edge is pointing from

Adjacency Matrix (cont.)

	0	1	2	3	4	5
0	F	T	T	F	T	F
1	T	F	F	T	F	T
2	T	F	F	F	T	F
3	F	T	F	F	F	F
4	T	F	T	F	F	T
5	F	T	F	F	T	F

Example:

Node 1 points to Node 2 (set to T)

Node 2 also points to Node 1 for undirected graph

Adjacency Matrix (cont.)

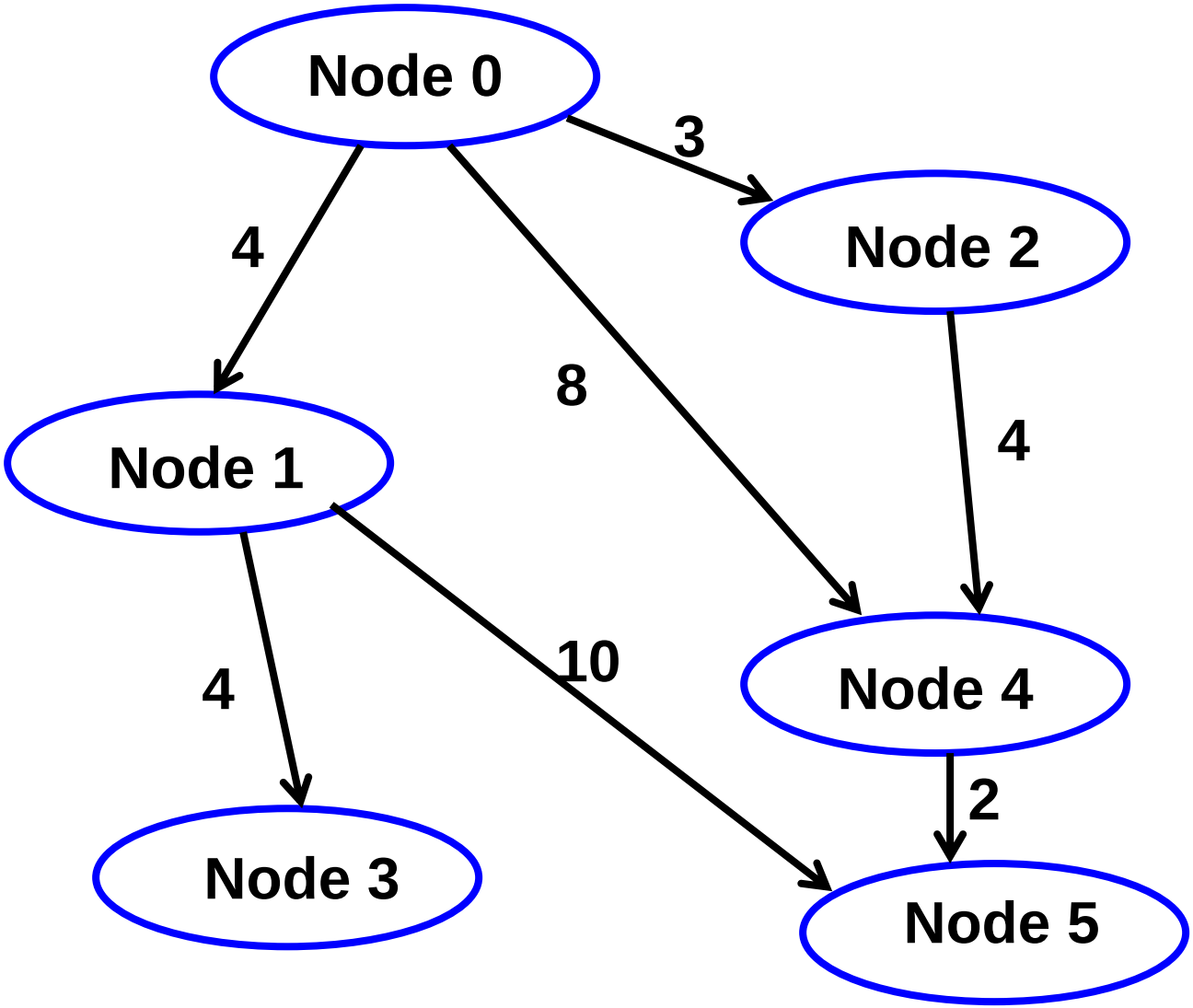
	0	1	2	3	4	5
0	∞	4	3	∞	8	∞
1	4	∞	∞	4	∞	10
2	3	∞	∞	∞	4	∞
3	∞	4	∞	∞	∞	∞
4	8	∞	4	∞	∞	2
5	∞	10	∞	∞	2	∞

Undirected weighted graph

∞ - Unconnected

Do NOT use 0 or symbol to denote unconnected.

Directed Weighted Graph



Adjacency Matrix (cont.)

	0	1	2	3	4	5
0	F	T	T	F	T	F
1	F	F	F	T	F	T
2	F	F	F	F	T	F
3	F	F	F	F	F	F
4	F	F	F	F	F	T
5	F	F	F	F	F	F

Directed graph

Example:

Node 0 points to Node 1 (set to T)

But Node 1 doesn't point to Node 0 (set to F)

Adjacency Matrix (cont.)

	0	1	2	3	4	5
0	∞	4	3	∞	8	∞
1	∞	∞	∞	4	∞	10
2	∞	∞	∞	∞	4	∞
3	∞	∞	∞	∞	∞	∞
4	∞	∞	∞	∞	∞	2
5	∞	∞	∞	∞	∞	∞

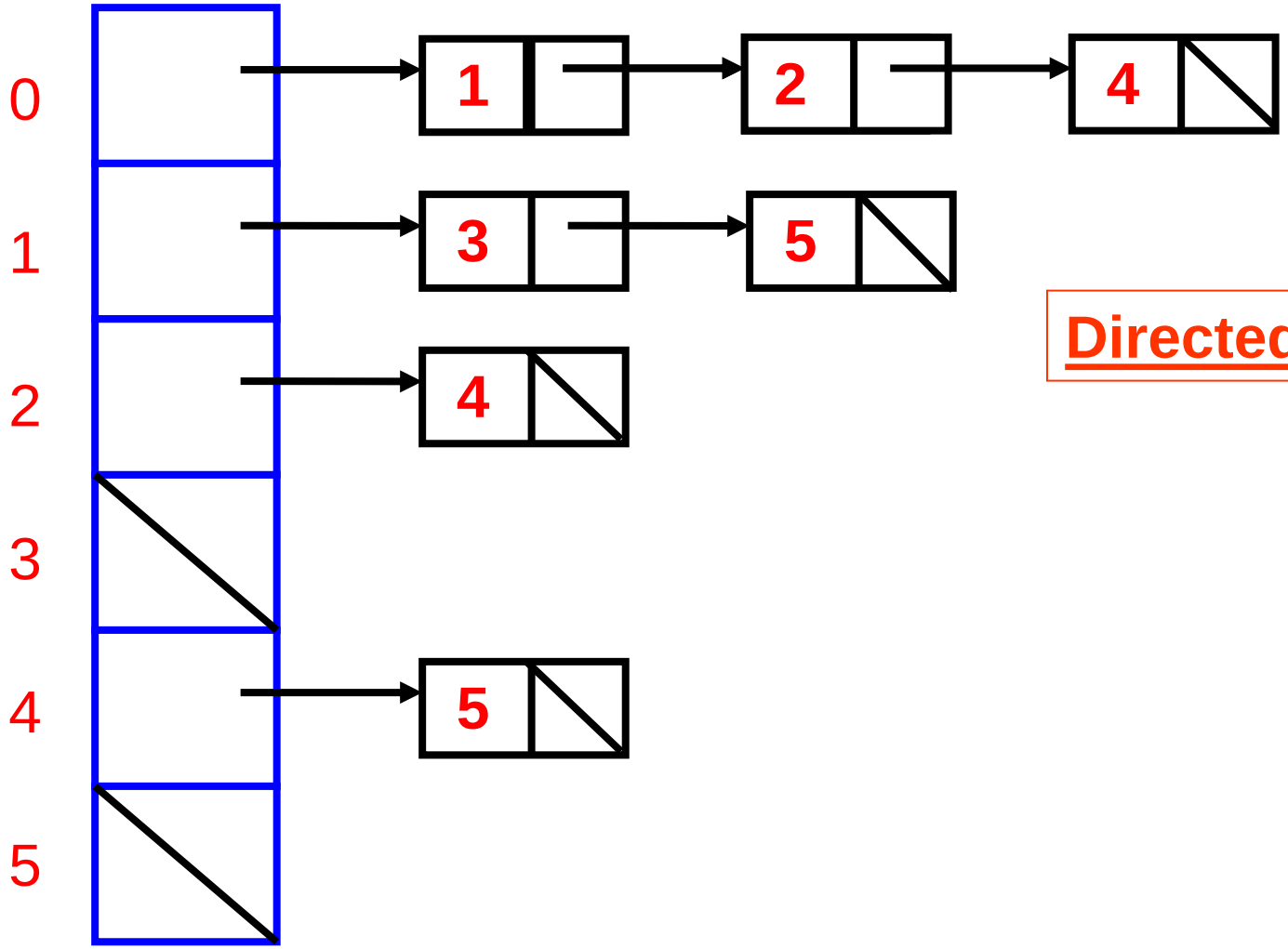
Directed weighted graph

∞ - Unconnected

Adjacency List

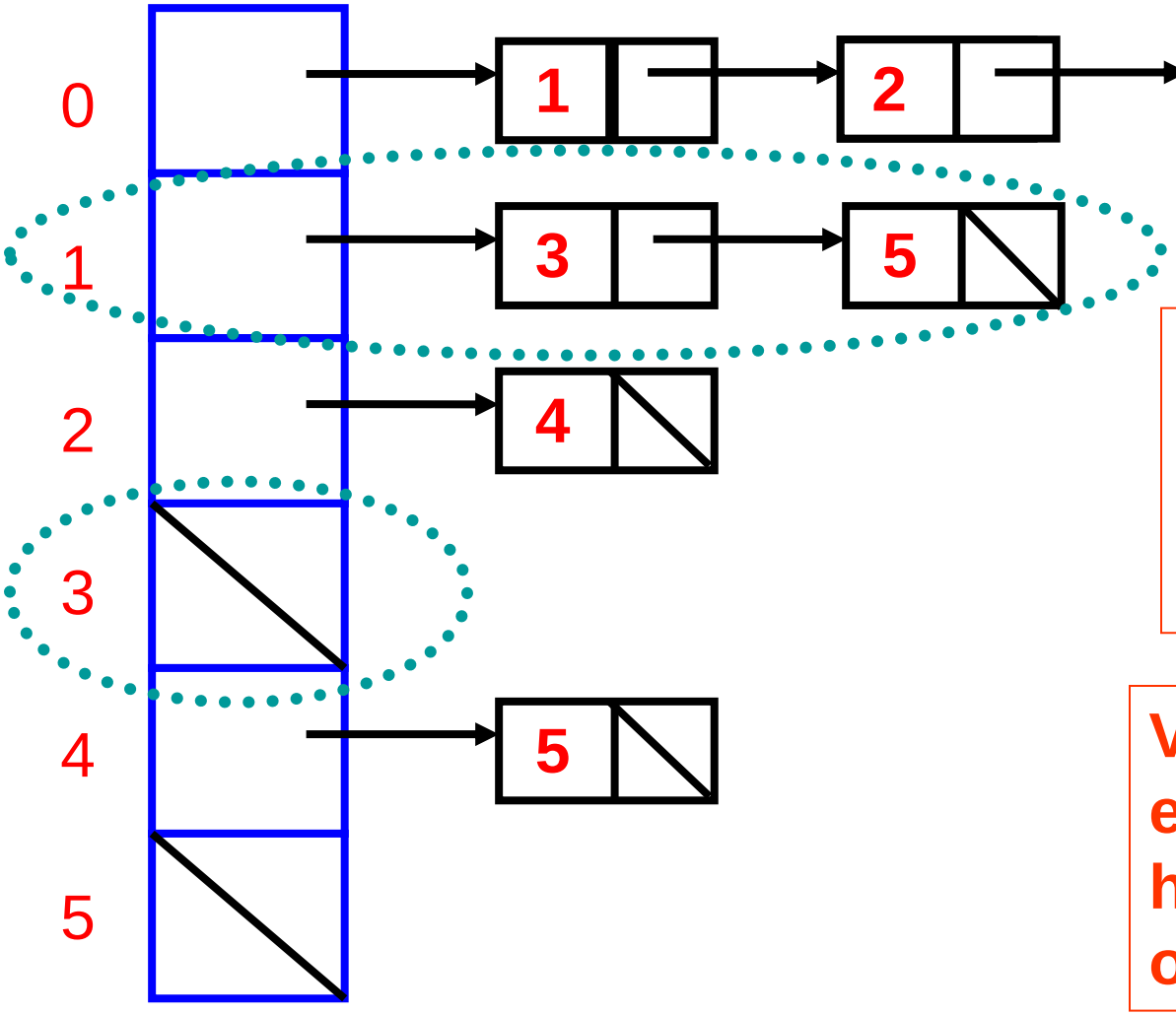
- An adjacency list is an array of linked lists
- The vertices are numbered 0 through n – 1
- Each index in the array corresponds to the number of a vertex
- The vertex (with an index number) is adjacent to every node in the linked list at that index

Adjacency List (cont.)



Directed graph

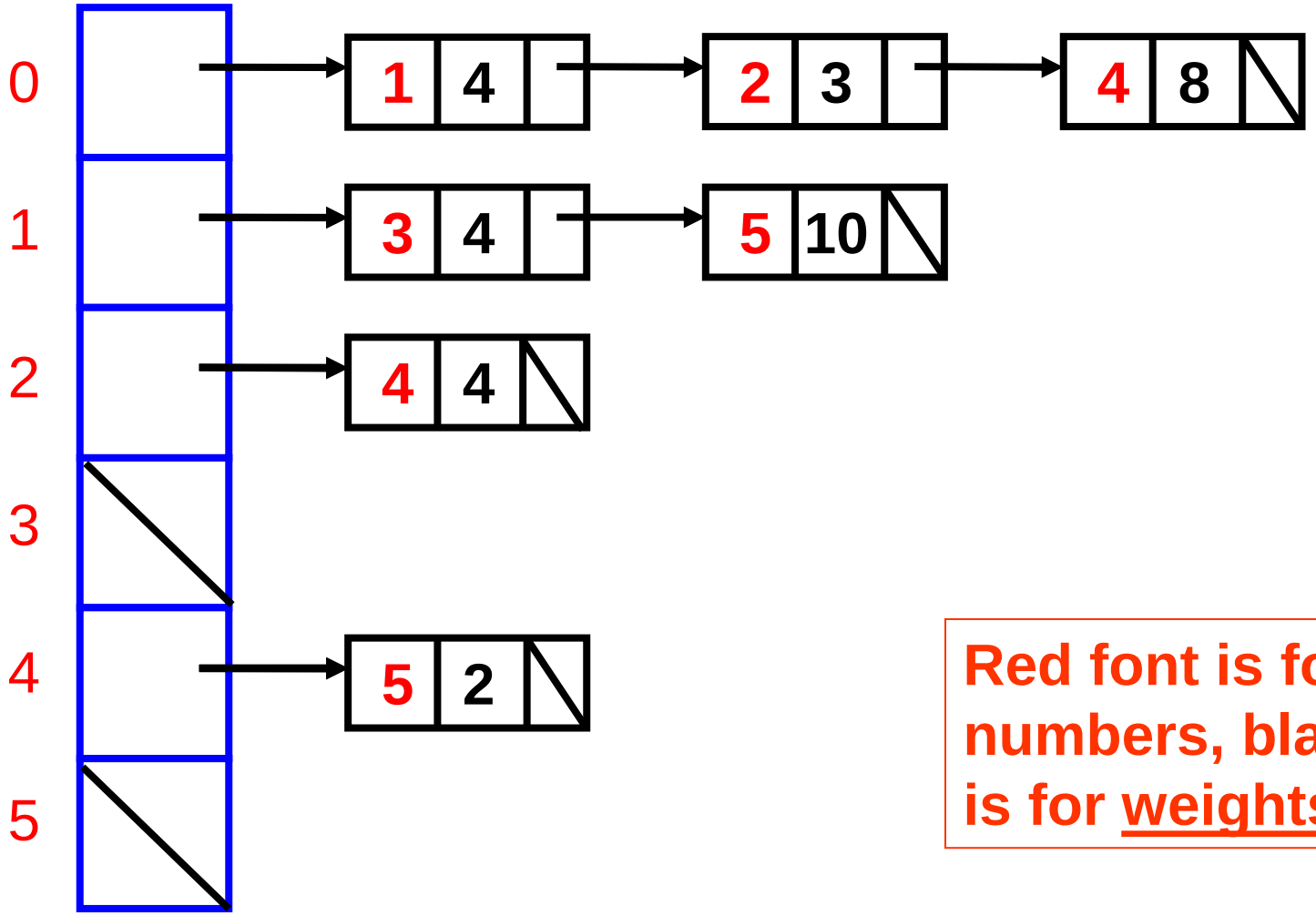
Adjacency List (cont.)



Vertex 1 has a link to vertex 3, and vertex 1 also has a link to vertex 5

Vertex 3 has an empty linked list – it has no link to any other vertices

Adjacency List for Directed Weighted Graph



Red font is for vertex numbers, black font is for weights

Adjacency Matrix vs. Adjacency List

- The speed of the adjacency matrix or the adjacency list depends on the algorithm
  - some algorithms need to know if there is a direct connection between two specific vertices – the adjacency matrix would be faster
  - some algorithms are written to process the linked list of an adjacency list, node by node – the adjacency list would be faster

Adjacency Matrix vs. Adjacency List (cont.)

- When both seem equally fast for a certain algorithm, then we consider memory usage
- We will consider the ***space complexity*** of each implementation
- Space complexities are like time complexities, but they tell us the effects on memory space usage as the problem size is varied

Adjacency Matrix vs. Adjacency List (cont.)

- An array of vertex is used for both implementations, which is $\Theta(n)$
- In the adjacency matrix, each dimension is n, so the spatial complexity is $\Theta(n^2)$

Adjacency Matrix vs. Adjacency List (cont.)

- In the adjacency list, there are n elements (vertices) in the array, giving a spatial complexity of $\Theta(n)$ for the array
- Each edge corresponds to a linked list node in the adjacency list
- Hence, the spatial complexity is O(n + m) where m is the total edges.

Adjacency Matrix vs. Adjacency List (cont.)

- The spatial complexity of the adjacency list really depends on whether the graph is ***sparse*** or ***dense***
- A sparse graph does not have that many edges relative to the number of vertices – if m is less than or equal to n, then the spatial complexity of the adjacency list is $\Theta(n)$
- In a dense graph, m may be close to n^2 edges, and then the spatial complexity of the adjacency list is $\Theta(n^2)$, the same as that for the adjacency matrix